Week 10: Version Control

Ellen Bledsoe

Version Control

In this week's lesson, we will learn how to keep a complete history of our work using `git` and GitHub.

Last week, we focused on making our code run anywhere. This week, we are adding a second piece to reproducibility: keeping track of *how our files changed over time* and sharing them with other people (including our future selves).

What Is Version Control?

Version control is a system that records changes to documents, code, data, etc. through time. It lets us take snapshots of our files (called *commits*) and go back to any earlier snapshot if we want to. We never have to save a copy called `analysis_FINAL_v2` again.

Version control also helps us work *collaboratively*. It keeps track of who changed what and notices when two people have changed the same line of code.

git vs. GitHub

These two names get used almost interchangeably, but they are different things.

`git` is the software that lives on your computer and does the tracking. GitHub is a website that stores a copy of your project in the cloud. A project that is tracked by `git` is called a *repository* (or "repo" for short).

Because there are two places where a repo lives, we need words for each:

- *Local*: the copy on your computer
- *Remote*: the copy on GitHub

Everything we do takes place in a short cycle. On our own computer, we save our files, *add* (or "stage") the ones we want to record, and *commit* them with a short message describing what we did. Then we sync with GitHub. We *push* our commits up to the remote and *pull* down any commits that someone else (or another computer of ours) has pushed.

```
Remote (GitHub)
      ^   |
 push |   | pull
      |   v
Local (your computer):  save -> add -> commit
```

Commit often and push when you have finished a chunk of work. Pull *before* you start working and again *before* you push, which saves us from most problems later on.

Getting Set Up

We only need to do the steps in this section once per computer.

Introduce Yourself to git

First, we need to tell `git` who we are. Use the same email address that you used for your GitHub account.

```
#install.packages(usethis)
library(usethis)

use_git_config(user.name = "Your Name", user.email = "your_email@arizona.edu")
```

We also need to change one `git` setting so that merging works smoothly later in the lesson. Open the `Terminal` tab (next to the `Console`) and run the line below. Without it, newer versions of `git` will refuse to pull when two people have changed the same repo, with an error about "divergent branches."

```
git config --global pull.rebase false
```

Make a Personal Access Token

When RStudio talks to GitHub, GitHub needs to know that it is really you. GitHub no longer accepts your account password for this. Instead, we create a "personal access token," which is like a special password that works for one computer.

Make sure you are logged into GitHub in your web browser, then run the following code.

```
usethis::create_github_token()
```

This opens a page on GitHub.

1. Leave the pre-selected scopes as they are.
2. Choose no expiration date. GitHub will try to convince you otherwise, but ignore it!
3. Click `Generate token`.
4. Copy the token. You will not be able to see it again!

Now we give the token to `git` so it can use it for us.

```
gitcreds::gitcreds_set()
```

When R asks for your password, paste in the *token* (not your GitHub password).

If RStudio ever asks for a password later or tells you that authentication failed, repeat these steps with a new token.

Think of the token like a password: don't share it or put it in a file that you commit!

Create a Repo and Connect It to RStudio

Now we need a repo. We will create it on GitHub, and then *clone* it, which makes a local copy that remembers where it came from. RStudio can do this for us when we create a new RStudio Project, as long as we choose `Version Control` and then `Git` instead of the usual `New Directory`.

The repo that we create on GitHub will have two files already in it:

- a `README` file, which is the "front page" of your repo on GitHub
- a `.gitignore` file, which lists files that `git` should *not* keep track of. The R template includes things like `.Rhistory` and `.RData`, which you may remember from our Week 9 conversation about not saving your workspace.

  Work on Question 1 in the Assignment

When your project opens, you should have a new `Git` tab in the upper right pane.

If it is missing, `usethis::git_sitrep()` will print a report of your `git` and GitHub setup that can help us figure out what went wrong.

Our First Commits

Commit the Data

Download `Gaeta_etal_CLC_data.csv` from Brightspace and move it into your new project folder. This file has measurements of fish length and scale length from several lakes.

Look at the `Git` tab. The data file is listed with a yellow `?`, which means that it is in our project folder but `git` is not tracking it yet. (You might also see a `.Rproj` file. Commit that on its own first, with a message like `Add RStudio Project file`, so that anyone who clones your repo can open it as an RStudio Project.)

To commit a file, we first need to *stage* it, which tells `git` that we want it included in our next commit.

1. Click the `Commit` button in the `Git` tab.
2. Check the box next to `Gaeta_etal_CLC_data.csv` to stage it.
3. Write a commit message in the box on the right.
4. Click `Commit`.

We want the message to be short but descriptive, written as if completing the sentence "This commit will…". Something like `Add fish size and growth rate data` is perfect. Messages like `update` or `stuff` will not help you at all when you look through your history in six months.

Click the `History` button (the clock). We now have one commit with our data file. Changes to a data file like this are too large to be useful to look at, but the history is there!

Commit Some Code

Let's start a Quarto file to work with our data. Create a new Quarto document, save it as `fish_analysis.qmd` in the main project folder, and add a code chunk to read in the data.

```
library(tidyverse)

fish_data <- read_csv("Gaeta_etal_CLC_data.csv")
```

Save the file and it will show up in the `Git` tab. Stage it, and commit it with the message `Start fish analysis script`.

If you click on the file in the commit window, you can see the `Diff`, which shows exactly what has changed since the last commit. Here, the whole file is new, so every line is added.

Building a History

Once a file has been committed and we haven't changed it since, it disappears from the `Git` tab. As soon as we make a change and save, it shows up again with an `M` for "modified".

Let's take a look at our data. Add a new code chunk to `fish_analysis.qmd`.

```
glimpse(fish_data)
```

Save the file, stage it, and commit it with the message `Look at the fish data`. In the `History`, each commit of `fish_analysis.qmd` shows only what we changed in that commit.

Next, let's find the smallest and largest scale length in each lake.

```
fish_data |>
  group_by(lakeid) |>
  summarize(min_scale_length = min(scalelength, na.rm = TRUE),
            max_scale_length = max(scalelength, na.rm = TRUE))
```

Save, stage, and commit with `Add scale length range by lake`.

Now let's add the average scale length, too.

```
fish_data |>
  group_by(lakeid) |>
  summarize(min_scale_length = min(scalelength, na.rm = TRUE),
            max_scale_length = max(scalelength, na.rm = TRUE),
            mean_scale_length = mean(scalelength, na.rm = TRUE))
```

Save, stage, and commit with `Add mean scale length by lake`. This time, the diff has green lines for what we added and red lines for what we removed. Notice that the `max_scale_length` line shows up as both, even though all we did to it was add a comma! `git` works line by line, so a changed line shows up as the old version deleted and the new version added.

Work on Question 2 in the Assignment

Committing Multiple Files

A commit can include as many files as we want. Let's use that to tidy up. Last week, we learned that raw data belongs in a `data_raw` folder, so let's move our data file there.

In the `Files` pane, create a new folder called `data_raw`. Then check the box next to `Gaeta_etal_CLC_data.csv` -> `More` -> `Move...` -> `data_raw`. Our code needs to know about the move, too.

```
fish_data <- read_csv("data_raw/Gaeta_etal_CLC_data.csv")
```

Look at the `Git` tab. `fish_analysis.qmd` has an `M`, the original data file has a red `D` for "deleted", and there is a new, untracked `data_raw` folder. `git` thinks that we deleted the data file and made a brand new one!

Stage both the old and the new data files, and `git` figures out what really happened: the two files become one, marked with an `R` for "rename". Stage `fish_analysis.qmd` as well and commit them all together with `Move data file into data_raw folder`.

Git as a Time Machine

Because every commit is saved, we can try things without worrying about breaking anything. Let's make a change that we don't like. What if we group by individual fish instead of by lake?

```
  group_by(fish_id) |>
```

Save the file. We have not committed this change, so we can throw it away: in the `Git` tab, select `fish_analysis.qmd` -> `More` -> `Revert` -> `Yes`. The file goes right back to how it was in our last commit.

What if we already committed the change, or we want an older version of the code? In the `History`, select the commit that you want, then choose `View file @ ...` to see the file as it was at that point. From there, you can copy and paste the piece you need or save that version over your current file.

This works for deleted files, too. If we delete `fish_analysis.qmd` in the `Files` pane, the `Git` tab will show it as deleted, and `Revert` brings it right back.

GitHub Remotes

So far, everything has stayed on our own computer. To back up our work and share it, we sync with our remote repo on GitHub, which `git` calls `origin`.

Push

To send our commits to GitHub, click the `Push` button (green arrow pointing up) in the `Git` tab. Before we push, our commits only show up in our local `History`. After, they are also on GitHub.

Go to your repo in your web browser and refresh it. You should see all of your files, and if you click on the commits, you will see the entire history there, too!

> Work on Question 3 in the Assignment

Pull

The big advantage of a remote is easy collaboration. No more emailing files back and forth, wondering whether you have the most recent version!

While we've been talking, a collaborator has added a short description to the `README` file on GitHub. Right now, your `README` only has the name of your repo as a title.

To play the collaborator yourself, go to your repo on GitHub, open `README.md`, and click the pencil icon. Add a description under the title, then click `Commit changes...` and commit directly to the `main` branch.

```
# FirstnameLastname_DataWrangling

This repo holds my work for Data Wrangling in R.
```

To get their changes, click the `Pull` button (blue arrow pointing down) in the `Git` tab. When you open `README.md` from the `Files` pane, the new line is there!

> Work on Question 4 in the Assignment

Merges

What happens if two people make changes at the same time? If you and your collaborator edit *different* parts of a file, `git` can combine the changes on its own.

Let's say you add a section about the data to the bottom of your `README` in RStudio. You save, stage, and commit.

```
## Data

Fish length and scale length data from Gaeta et al.
```

Meanwhile, your collaborator decides that the title should be friendlier. They change the first line on GitHub and commit it.

```
# Data Wrangling in R
```

When you try to push, you get an error: the remote contains work that you do not have locally. That is `git` stopping us from overwriting our collaborator's work. The fix is to `Pull` first. The merge happens automatically, and we now have both sets of changes. The remote doesn't have our version yet, so we `Push` one more time.

Merge Conflicts

If you and your collaborator edit the *same* line, `git` cannot know whose change to keep. A human has to decide.

Let's say you decide to reword the description in your `README`, and you commit the change.

```
This repo holds my code and data for Data Wrangling in R.
```

Meanwhile, your collaborator changes that very same line on GitHub and commits it first.

```
This repo holds my work for Data Wrangling in R for Ecologists.
```

When you push, you get the same error as before. When you pull, though, we get a merge conflict. In the `Git` tab, the file is marked with a `U` for "unmerged". When we open the file, `git` has marked the problem for us.

```
<<<<<<< HEAD
This repo holds my code and data for Data Wrangling in R.
=======
This repo holds my work for Data Wrangling in R for Ecologists.
>>>>>>> 8d7ed087a1abeb59d5b896416a625048654e03f4
```

The first block (between `<<<<<<<` and `=======`) is your version, and the second block (between `=======` and `>>>>>>>`) is the version from the remote.

To fix it, edit the file so that only the text you want is left, and delete all of the marker lines. Here, we want to keep both changes.

```
This repo holds my code and data for Data Wrangling in R for Ecologists.
```

Save the file, stage it, and commit with a message that says this was a merge. The fixed version still isn't on GitHub, so finish up by pushing.

Conflicts feel scary the first time, but nothing is lost. Read the markers, decide what the file should say, and commit.

Your Final Project Repo

In the rest of the assignment, you will repeat this process for your final project. Questions 5 and 6 have you set up a repo, add your files, and write a `README` file, so that your project is on GitHub from here on out. Question 7 shows you how to share a rendered document there, too.

Helpful Resources

Below are some resources that you might find helpful as you get comfortable with `git` and GitHub.

- Happy Git with R: the resource I find most helpful
- GitHub Docs
- Jenny Bryan: Excuse me, do you have a moment to talk about version control?